

Hands-On Lab — Answer Key
Exercises 1-3 (Bell state, Deutsch-Jozsa, Grover)

```
pip install qiskit qiskit-aer -q
```

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

sim = AerSimulator()

def run(qc, shots=1000):
    return sim.run(transpile(qc, sim), shots=shots).result().get_counts()
```

Double-click (or enter) to edit

▼

EXERCISE 1 — $|\Psi\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$ instead of $|\Phi\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$

Idea: $|\Phi\rangle$ already correlates the two qubits. Flipping ONE of them turns "same" into "different". One extra X gate is the whole answer.

$|\Phi\rangle \xrightarrow{\text{X on q1}} |\Psi\rangle$

```
qc1 = QuantumCircuit(2)
qc1.h(0)
qc1.cx(0, 1)
qc1.x(1)          # <-- the only new line vs. the demo
qc1.measure_all()

# Equivalent alternative (flip the target BEFORE entangling):
qc1_alt = QuantumCircuit(2)
qc1_alt.x(1)
qc1_alt.h(0)
qc1_alt.cx(0, 1)
qc1_alt.measure_all()
```

Expect: only "01" and "10", each ~50%. Never 00 or 11.

Teaching point: the entanglement (the H+CX) is untouched; X only relabels the correlation. Both Bell states are maximally entangled.

▼

EXERCISE 2 — Deutsch-Jozsa on 2 qubits (1 input qubit + 1 ancilla)

$f: \{0,1\} \rightarrow \{0,1\}$. Constant: $f(0)=f(1)$. Balanced: $f(0)\neq f(1)$.

Classically you must query f twice. Quantumly: ONE query.

Recipe: X on ancilla, H on both $\rightarrow$ oracle $\rightarrow$ H on input $\rightarrow$ measure input.

measure 0 $\Rightarrow$ constant

measure 1 $\Rightarrow$ balanced

```
def dj_oracle(kind):
    """Return a 2-qubit oracle U_f: |x>|y> -> |x>|y XOR f(x)>."""
    orc = QuantumCircuit(2)
    if kind == "constant":
        pass # f(x) = 0 for all x (identity)
        # orc.x(1) # f(x) = 1 for all x -- the other constant
    elif kind == "balanced":
        orc.cx(0, 1) # f(x) = x
        # orc.cx(0, 1); orc.x(1) # f(x) = NOT x -- the other balanced
    else:
        raise ValueError(kind)
    return orc

def deutsch_jozsa(kind):
    qc = QuantumCircuit(2, 1)
    qc.x(1) # ancilla -> |1>
    qc.h([0, 1]) # ancilla -> |->, input -> |+>
    qc.compose(dj_oracle(kind), inplace=True)
    qc.h(0)
    qc.measure(0, 0)
    return qc
```

Expect: constant $\rightarrow$ {'0': 1000}, balanced $\rightarrow$ {'1': 1000}. Deterministic.

Teaching point: PHASE KICKBACK. The ancilla in $|-\rangle$ turns U_f into a $(-1)^{f(x)}$ phase on the INPUT register; the final H turns that phase difference into a measurable bit. Nothing about $f(x)$ itself is learned — only the global property "constant vs balanced".

▼

EXERCISE 3 — Grover on 2 qubits ($N = 4$), 1 vs 3 iterations

Marked item: $|11\rangle$. Oracle = CZ (flips the sign of $|11\rangle$ only).

Diffuser = "reflect about the mean" = H X CZ X H sandwich.

```
def grover(n_iterations, marked="11"):
    qc = QuantumCircuit(2)
    qc.h([0, 1]) # uniform superposition over 4 states

    for _ in range(n_iterations):
        # ---- oracle: phase-flip the marked state ----
```

```

# X-wrap whichever qubits are 0 in `marked` so CZ hits the right corner.
# qiskit string order is little-endian: marked[::-1][q] is qubit q's bit.
bits = marked[::-1]
for q, b in enumerate(bits):
    if b == "0":
        qc.x(q)
qc.cz(0, 1)
for q, b in enumerate(bits):
    if b == "0":
        qc.x(q)

# ---- diffuser: inversion about the average ----
qc.h([0, 1])
qc.x([0, 1])
qc.cz(0, 1)
qc.x([0, 1])
qc.h([0, 1])

qc.measure_all()
return qc

```

✓ Expect: 1 iteration -> '11' with probability 1.000 (100%, exactly).

3 iterations -> '11' with probability 0.25 (same as random guessing).

Why: success amplitude after k iterations = $\sin((2k+1)*\theta)$, $\sin(\theta)=1/\sqrt{N}=1/2$
so $\theta = 30$ degrees.

$k=0 \rightarrow \sin(30) = 0.50 \rightarrow 25\%$

$k=1 \rightarrow \sin(90) = 1.00 \rightarrow 100\% \leftarrow$ optimal, $\text{floor}(\pi/4 * \sqrt{4}) = 1$

$k=2 \rightarrow \sin(150) = 0.50 \rightarrow 25\%$

$k=3 \rightarrow \sin(210) = -0.50 \rightarrow 25\%$

Teaching point: Grover is a ROTATION, not a ratchet. More iterations is not "more search" — you rotate past the target and amplitude drains back out.

This is the single most useful intuition on the slide: quantum speedups have a *correct* amount of work, and overshooting is a real failure mode.

```

if __name__ == "__main__":
    print("=" * 62)
    print("EX 1 – Bell state |Psi>")
    print("=" * 62)
    print("  H,CX,X(1) :", run(qc1))
    print("  X(1),H,CX :", run(qc1_alt))

    print()
    print("=" * 62)
    print("EX 2 – Deutsch-Jozsa (1 query)")
    print("=" * 62)
    for kind in ("constant", "balanced"):
        counts = run(deutsch_jozsa(kind))
        verdict = "constant" if max(counts, key=counts.get) == "0" else "balanced"
        print(f"  oracle={kind:9s} -> {str(counts):22s} verdict: {verdict}")

    print()
    print("=" * 62)
    print("EX 3 – Grover, marked = |11>")

```

```

print("=" * 62)
for k in (0, 1, 2, 3):
    counts = run(grover(k), shots=4000)
    p = counts.get("11", 0) / 4000
    tag = " <-- optimal" if k == 1 else ""
    print(f" {k} iteration(s): P(11) = {p:.3f}    {str(counts):48s}{tag}")

```

```
=====
EX 1 – Bell state |Psi+>
=====
```

```

H,CX,X(1) : {'10': 527, '01': 473}
X(1),H,CX : {'01': 500, '10': 500}

```

```
=====
EX 2 – Deutsch-Jozsa (1 query)
=====
```

```

oracle=constant -> {'0': 1000}          verdict: constant
oracle=balanced -> {'1': 1000}          verdict: balanced

```

```
=====
EX 3 – Grover, marked = |11>
=====
```

```

0 iteration(s): P(11) = 0.257    {'11': 1029, '01': 950, '10': 968, '00': 1053}
1 iteration(s): P(11) = 1.000    {'11': 4000}                                <-- optimal
2 iteration(s): P(11) = 0.234    {'11': 934, '10': 1029, '00': 1025, '01': 1012}
3 iteration(s): P(11) = 0.241    {'01': 1011, '10': 1000, '00': 1024, '11': 965}

```

Start coding or [generate](#) with AI.